	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 16
	Name: CH.PAVAN THEJA Reg. No:2503a52477
	Department: Computer Science and AISemester: I
	AI Assisted Coding-AssignmentDate: 22-09-2026

1. Question

– AI-Assisted Bubble Sort Analysis

Prompt
“Explain Bubble Sort step by step for the given list. Identify opportunities to reduce unnecessary comparisons. Do not write the complete program yet.”

Code : numbers = [64, 34, 25, 12, 22, 11, 90]

```
def bubble_sort(values):
    """Sort values with Bubble Sort and count comparisons and swaps."""
    result = values.copy()
    comparisons = 0
    swaps = 0

    for pass_number in range(len(result) - 1):
        for index in range(len(result) - 1 - pass_number):
            comparisons += 1
            if result[index] > result[index + 1]:
                result[index], result[index + 1] = result[index + 1], result[index]
                swaps += 1

    return result, comparisons, swaps

def bubble_sort_early_exit(values):
    """Sort values and stop when a complete pass makes no swaps."""
    result = values.copy()
    comparisons = 0
    swaps = 0

    for pass_number in range(len(result) - 1):
        swapped = False
        for index in range(len(result) - 1 - pass_number):
            comparisons += 1
            if result[index] > result[index + 1]:
```

```
                result[index], result[index + 1] = result[index + 1], result[index]
                swaps += 1
                swapped = True

        if not swapped:
            break

    return result, comparisons, swaps

def show_one_pass(values):
    """Display the comparisons and swaps from the first complete pass."""
    result = values.copy()
    actions = []

    for index in range(len(result) - 1):
        left = result[index]
        right = result[index + 1]
        if left > right:
#           result[index], result[index + 1] = right, left
#           actions.append(f"swap {left} and {right} -> {result}")
#       else:
#           actions.append(f"keep {left} and {right} -> {result}")

#   print("One complete Bubble Sort pass:")
#   for action in actions:
#       print(f"  {action}")
#   print(f"  Result after pass: {result}")

# def compare_versions(test_lists):
#   for test_list in test_lists:
#       original = bubble_sort(test_list)
#       improved = bubble_sort_early_exit(test_list)
#       print(f"\nInput: {test_list}")
#       print(f"Original: {original[0]}, comparisons={original[1]}, swaps={original[2]}")
#       print(f"Early exit: {improved[0]}, comparisons={improved[1]},
swaps={improved[2]}")

# if __name__ == "__main__":
#   test_lists = [
#       numbers,
#       [1, 2, 3, 4, 5],
#       [5, 1, 4, 2, 8],
#   ]

#   show_one_pass(numbers)
```

```
# print("\nEarly-exit optimisation: stop after a pass with no swaps.")
# compare_versions(test_lists)
```

Output

```
Input: [64, 34, 25, 12, 22, 11, 90]
    Original: [11, 12, 22, 25, 34, 64, 90], comparisons=21, swaps=14

    Early exit: [11, 12, 22, 25, 34, 64, 90], comparisons=21, swaps=14


    Input: [1, 2, 3, 4, 5]

    Original: [1, 2, 3, 4, 5], comparisons=10, swaps=0

    Early exit: [1, 2, 3, 4, 5], comparisons=4, swaps=0


    Input: [5, 1, 4, 2, 8]

    Original: [1, 2, 4, 5, 8], comparisons=10, swaps=4

    Early exit: [1, 2, 4, 5, 8], comparisons=9, swaps=4

    Observation: Bubble Sort successfully sorts the given list in ascending order
    by repeatedly comparing and swapping adjacent elements. The early-exit
    optimization reduces unnecessary comparisons when the list becomes
    sorted.
```

#----------2 question:

Linear Search vs Binary Search

Prompt: “Compare Linear Search and Binary Search for the given sorted list [5, 12, 18, 23, 31, 45, 56, 72, 89]. Explain why Binary Search requires sorted data, count the comparisons for both methods, and test first, middle, last, and absent elements. Suggest one valid optimization.”

Code:

```
earch_numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
search_target = 45

def linear_search(values, target):
    """Find target from left to right and count inspected elements."""
    comparisons = 0
    for index, value in enumerate(values):
        comparisons += 1
        if value == target:
            return index, comparisons
    return -1, comparisons
```

```
def binary_search(values, target):
    """Find target in sorted values by repeatedly checking the middle."""
    left = 0
    right = len(values) - 1
    comparisons = 0

    while left <= right:
        middle = (left + right) // 2
        comparisons += 1
        if values[middle] == target:
            return middle, comparisons
        if values[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1, comparisons

def compare_searches(values, targets):
    for target in targets:
        linear_result = linear_search(values, target)
        binary_result = binary_search(values, target)
        print(f"\nTarget: {target}")
        print(
            f"Linear Search: index={linear_result[0]}, "
            f"comparisons={linear_result[1]}"
        )
        print(
            f"Binary Search: index={binary_result[0]}, "
            f"comparisons={binary_result[1]}"
        )

if __name__ == "__main__":
    print("\nQ2 - Linear Search vs Binary Search")
    print("Binary Search needs sorted data so it can discard half the remaining")
    print("values after each middle-element comparison.")
    compare_searches(search_numbers, [search_numbers[0], search_numbers[4],
search_numbers[-1], 100])
    print("\nValid optimisation: for many repeated lookups, keep the data sorted and")
    print("use Binary Search, reducing each lookup from O(n) to O(log n).")
```

output: **Q2 -** Linear Search vs Binary Search
Binary Search needs sorted data so it can discard half the remaining
values after each middle-element comparison.

Target: 5
Linear Search: index=0, comparisons=1
Binary Search: index=0, comparisons=3

Target: 31
Linear Search: index=4, comparisons=5
Binary Search: index=4, comparisons=1

Target: 89
Linear Search: index=8, comparisons=9
Binary Search: index=8, comparisons=4

Target: 100
Linear Search: index=-1, comparisons=9
Binary Search: index=-1, comparisons=4

Observation: Linear Search checks elements sequentially, while Binary Search repeatedly divides the sorted list into halves.
Binary Search generally requires fewer comparisons, especially for larger sorted datasets.
The choice depends on whether the data is sorted and how frequently searching is performed.

3 question: Compare Sorting Algorithms
Prompt : " Compare Bubble Sort, Selection Sort, and Insertion Sort using the given list [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]. Count their comparisons and swaps/shifts, explain their strengths and weaknesses, and identify which algorithm is most suitable for nearly sorted data. Compare the results with their theoretical time complexities."
Code:
```
def q3_bubble_sort(values):
    result = values.copy()
    comparisons = 0
    swaps = 0
    for pass_number in range(len(result) - 1):
        for index in range(len(result) - 1 - pass_number):
            comparisons += 1
            if result[index] > result[index + 1]:
                result[index], result[index + 1] = result[index + 1], result[index]
                swaps += 1
    return result, comparisons, swaps

def selection_sort(values):
    result = values.copy()
    comparisons = 0
    swaps = 0
```

```
    for start in range(len(result) - 1):
        minimum_index = start
        for index in range(start + 1, len(result)):
            comparisons += 1
            if result[index] < result[minimum_index]:
                minimum_index = index
        if minimum_index != start:
            result[start], result[minimum_index] = result[minimum_index], result[start]
            swaps += 1
    return result, comparisons, swaps

def insertion_sort(values):
    result = values.copy()
    comparisons = 0
    shifts = 0
    for index in range(1, len(result)):
        current = result[index]
        position = index - 1
        while position >= 0:
            comparisons += 1
            if result[position] <= current:
                break
            result[position + 1] = result[position]
            shifts += 1
            position -= 1
        result[position + 1] = current
    return result, comparisons, shifts

def compare_sorting_algorithms(values):
    algorithms = [
        ("Bubble Sort", q3_bubble_sort, "swaps"),
        ("Selection Sort", selection_sort, "swaps"),
        ("Insertion Sort", insertion_sort, "shifts"),
    ]
    print(f"\nInput used for every algorithm: {values}")
    print("Algorithm      Comparisons  Swaps/Shifts  Sorted result")
    for name, algorithm, movement_name in algorithms:
        result, comparisons, movements = algorithm(values)
        print(f"{name:<16}{comparisons:<13}{movements:<14}{result}")

if __name__ == "__main__":
    print("\nQ3 - Sorting algorithm analysis")
    compare_sorting_algorithms(sorting_data)
    print("\nStrengths and weaknesses:")
    print("- Bubble Sort is simple, but usually makes many comparisons and swaps.")
```

```
    print("- Selection Sort makes few swaps, but still makes O(n^2) comparisons.")
    print("- Insertion Sort is simple and efficient for small or nearly sorted data.")
    print("\nNearly sorted data: Insertion Sort is usually the best choice here.")
    print("All three algorithms are O(n^2) in the average and worst cases;")
    print("Insertion Sort can be O(n) when the data is already nearly sorted.")
```

output:

```
    Input used for every algorithm: [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
Algorithm       Comparisons  Swaps/Shifts  Sorted result
Bubble Sort     45           24            [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Selection Sort  45           6             [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
Insertion Sort  31           24            [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
```

Strengths and weaknesses:
- Bubble Sort is simple, but usually makes many comparisons and swaps.
- Selection Sort makes few swaps, but still makes O(n^2) comparisons.
- Insertion Sort is simple and efficient for small or nearly sorted data.

Nearly sorted data: Insertion Sort is usually the best choice here.
All three algorithms are O(n^2) in the average and worst cases;
Insertion Sort can be O(n) when the data is already nearly sorted.

Observation : All three algorithms successfully sort the same input list, but their comparisons and swaps/shifts differ.
Insertion Sort is generally more efficient for nearly sorted data because it requires fewer shifts.
The experimental results can be compared with the theoretical **O(n²)** time complexity of these algorithms.

4 question: – Optimising a Student Search and Sort Program
Prompt : “Analyze the student records [("Arun",82), ("Bala",65), ("Chitra",91), ("Divya",74), ("Esha",88)]. Create functions to find the highest mark, sort students by marks, and search by name. Identify unnecessary operations and suggest an efficient approach, then compare the time complexity of the original and improved solutions.”
Code:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88),
]

def highest_mark_basic(student_list):
    if not student_list:
        return None
    current_highest = student_list[0]
```

```
    for student in student_list[1:]:
        if student[1] > current_highest[1]:
            current_highest = student
    return current_highest

def sort_students_basic(student_list):
    return sorted(student_list, key=lambda student: student[1], reverse=True)

def find_student_basic(student_list, name):
    for student in student_list:
        if student[0] == name:
            return student
    return None

def straightforward_student_report(student_list, name):
    """Run each basic operation independently with a linear name search."""
    return {
        "highest": highest_mark_basic(student_list),
        "sorted": sort_students_basic(student_list),
        "found": find_student_basic(student_list, name),
    }

def build_student_index(student_list):
    """Compute the highest mark and build a name index in one scan."""
    if not student_list:
        return None, {}

    highest = student_list[0]
    by_name = {}
    for student in student_list:
        by_name[student[0]] = student
        if student[1] > highest[1]:
            highest = student
    return highest, by_name

def sort_students_improved(student_list):
    return sorted(student_list, key=lambda student: student[1], reverse=True)

def find_student_improved(student_index, name):
    return student_index.get(name)

def improved_student_report(student_list, name):
    highest, by_name = build_student_index(student_list)
    return {
        "highest": highest,
```

```
        "sorted": sort_students_improved(student_list),
        "found": find_student_improved(by_name, name),
    }

if __name__ == "__main__":
    print("\nQ4 - Student search and sort")
    basic = straightforward_student_report(students, "Chitra")
    improved = improved_student_report(students, "Chitra")
    print(f"Basic highest mark: {basic['highest']}")
    print(f"Basic sorted students: {basic['sorted']}")
    print(f"Basic name search: {basic['found']}")
    print(f"Improved highest mark: {improved['highest']}")
    print(f"Improved sorted students: {improved['sorted']}")
    print(f"Improved name search: {improved['found']}")
    print("\nOptimisation: avoid scanning the list again for every repeated name search;")
    print("build a dictionary once, changing each later search from O(n) to O(1) average.")
    print("Both versions sort in O(n log n), while the one-pass highest-mark scan is O(n).")
    print("The improved approach uses O(n) extra space for the name index; the basic")
    print("approach uses O(1) extra search space and is fine for occasional lookups.")
```

output : Basic highest mark: ('Chitra', 91)
Basic sorted students: [('Chitra', 91), ('Esha', 88), ('Arun', 82), ('Divya', 74), ('Bala', 65)]
Basic name search: ('Chitra', 91)
Improved highest mark: ('Chitra', 91)
Improved sorted students: [('Chitra', 91), ('Esha', 88), ('Arun', 82), ('Divya', 74), ('Bala', 65)]
Improved name search: ('Chitra', 91)

Optimisation: avoid scanning the list again for every repeated name search;
build a dictionary once, changing each later search from O(n) to O(1) average.
Both versions sort in O(n log n), while the one-pass highest-mark scan is O(n).
The improved approach uses O(n) extra space for the name index; the basic
approach uses O(1) extra search space and is fine for occasional lookups.
Observation : The program successfully finds the highest mark, sorts the students, and searches for a student by name.
The improved approach can reduce unnecessary repeated operations by using suitable data structures and efficient searching.
The original and improved solutions can be compared based on their time complexity and number of operations.
5 question: Optimisation Challenge: Best Combination
Prompt : “For the given items and budget of ₹50,000, find the combination of items with the maximum total price without exceeding the budget. Compare Brute Force, Greedy, and Dynamic Programming approaches, identify which method guarantees an optimal solution, and explain its time and space complexity. Test the solution with at least three different budgets.”

Code : items = [
("Laptop", 60000),
("Phone", 30000),
("Tablet", 20000),
("Watch", 10000),
("Headphones", 5000),
]

budget = 50000

def brute_force_combination(item_list, maximum_budget):
"""Try every subset and return the most valuable valid subset."""
best_items = []
best_total = 0

def search(index, chosen_items, total):
nonlocal best_items, best_total
if index == len(item_list):
if total > best_total:
best_items = chosen_items.copy()
best_total = total
return

search(index + 1, chosen_items, total)
name, price = item_list[index]
if total + price <= maximum_budget:
chosen_items.append(name)
search(index + 1, chosen_items, total + price)
chosen_items.pop()

search(0, [], 0)
return best_items, best_total

def greedy_combination(item_list, maximum_budget):
"""Choose the most expensive remaining item that fits each time."""
chosen_items = []
total = 0
for name, price in sorted(item_list, key=lambda item: item[1], reverse=True):
if total + price <= maximum_budget:
chosen_items.append(name)
total += price
return chosen_items, total

def dynamic_programming_combination(item_list, maximum_budget):
"""Solve the 0/1 knapsack problem and reconstruct an optimal subset."""

```
    item_count = len(item_list)
    dp = [[0] * (maximum_budget + 1) for _ in range(item_count + 1)]

    for item_index in range(1, item_count + 1):
        price = item_list[item_index - 1][1]
        for current_budget in range(maximum_budget + 1):
            dp[item_index][current_budget] = dp[item_index - 1][current_budget]
            if price <= current_budget:
                dp[item_index][current_budget] = max(
                    dp[item_index][current_budget],
                    dp[item_index - 1][current_budget - price] + price,
                )

    chosen_items = []
    current_budget = maximum_budget
    for item_index in range(item_count, 0, -1):
        if dp[item_index][current_budget] != dp[item_index - 1][current_budget]:
            name, price = item_list[item_index - 1]
            chosen_items.append(name)
            current_budget -= price

    chosen_items.reverse()
    return chosen_items, dp[item_count][maximum_budget]

def compare_combinations(item_list, budgets):
    for current_budget in budgets:
        brute_force = brute_force_combination(item_list, current_budget)
        greedy = greedy_combination(item_list, current_budget)
        optimal = dynamic_programming_combination(item_list, current_budget)
        print(f"\nBudget: {current_budget}")
        print(f"Brute Force: {brute_force}")
        print(f"Greedy: {greedy}")
        print(f"Dynamic Programming: {optimal}")

if __name__ == "__main__":
    print("\nQ5 - Best combination under a budget")
    compare_combinations(items, [budget, 35000, 15000])
    print("\nBrute Force always finds the optimum but takes O(2^n) time.")
    print("Greedy takes O(n log n) after sorting, but it is not guaranteed optimal")
    print("for 0/1 knapsack problems because an early choice can block a better set.")
    print("Dynamic Programming guarantees the optimum in O(n * budget) time and")
    print("O(n * budget) space here. For larger inputs, a 1D DP array reduces space")
    print("to O(budget), or item values can be scaled when prices are very large.")
```

out put : Brute Force always finds the optimum but takes O(2^n) time.

Greedy takes O(n log n) after sorting, but it is not guaranteed optimal for 0/1 knapsack problems because an early choice can block a better set. Dynamic Programming guarantees the optimum in O(n * budget) time and O(n * budget) space here. For larger inputs, a 1D DP array reduces space to O(budget), or item values can be scaled when prices are very large.

Observation : The goal is to maximize the total item value without exceeding the given budget.
Brute Force and Dynamic Programming can guarantee an optimal solution, while Greedy does not always guarantee the best combination.
Dynamic Programming is more efficient than Brute Force for larger inputs, but requires additional memory.